

Sentiment Analysis using Machine Learning Classifiers



Supervised By -

Mr. Biswajit Senapati
Department of Computer Science
& Engineering

Presented By -

Deepthi - 423192
Pavan - 423167



Presentation Outline

- Introduction
- Literature Review
- Problem Statement
- Objective
- Methodology Used
- Key Findings While Preprocessing
- Explanation of Code



Introduction

Machine Learning

Machine Learning (ML) is a branch of **Artificial Intelligence (AI)** that enables computers to **learn from data and make predictions** without being explicitly programmed . Instead of using **hardcoded rules**, ML models **identify patterns** in data and improve their performance over time.

Types of Machine Learning Models

Linear Regression, Logistic Regression, Decision Trees, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), K-Means Clustering, Principal Component Analysis (PCA), Autoencoders, Hierarchical Clustering, Gaussian Mixture Model (GMM)

Sentiment Analysis

- Sentiment Analysis is a **Natural Language Processing (NLP) technique** that determines the **emotional tone** of a text.
- It classifies text as **positive, negative, or neutral** based on opinions expressed.



Literature Review

Title	Models	Advantages	Disadvantages	Result
<i>Optimization of Sentiment Analysis Using Machine Learning Classifiers</i>	SVM, Naïve Bayes, Random Forest, Logistic Regression	Improved accuracy by optimizing feature selection and preprocessing	Might have higher computational cost	OneR classifier achieved the highest precision (91.3%) and F-measure (97%).
<i>: A Comparative Study of Sentiment Analysis Using NLP and Different Machine Learning Techniques on US Airline Twitter Data</i>	NLP-based approaches (LSTMs, CNNs, SVM, Naïve Bayes)	Comparative analysis across multiple models provides insights into the best-performing model	May require large datasets for effective training	SVM and Logistic Regression with Bag-of-Words technique provided 77% accuracy.
<i>Sentiment Analysis Based on Machine-Learning Classifiers for Datasets</i>	traditional classifiers (SVM, Decision Trees, Naïve Bayes, KNN)	Evaluated different classifiers on datasets	Some models may not generalize well across different datasets	best classifiers for different types of textual data.



Problem Statement

- Millions of social media posts are created daily, containing opinions on brands, government policies, products, and social issues.
- Businesses and organizations struggle to manually analyze public sentiment due to the **massive volume** of data.
- **Lack of automated systems** for real-time sentiment detection and trend analysis.
- Existing systems often **fail to detect sarcasm, contextual meaning, and code-mixed language** used in informal social media text.



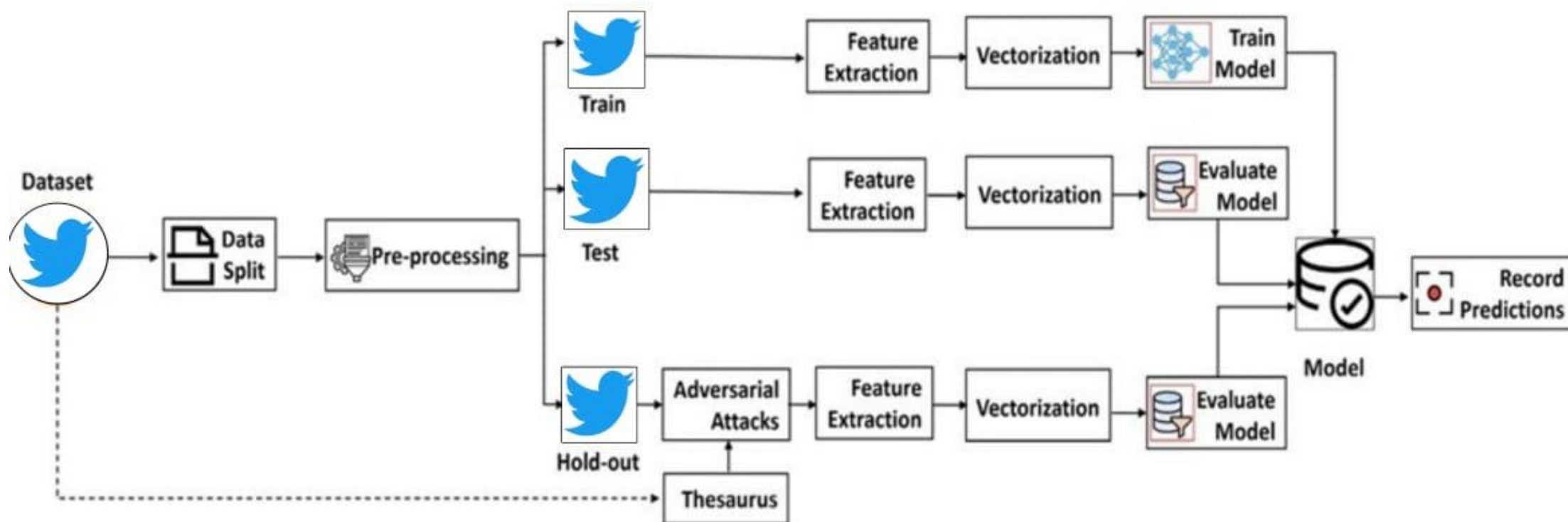
Objective

To develop a **Machine Learning-based** system that can:

- Automatically classify social media posts into **positive, negative, and neutral** sentiment categories.
- Detect **trending topics** based on real-time sentiment shifts.
- Improve sentiment detection by handling **slang, emojis, and sarcasm**.

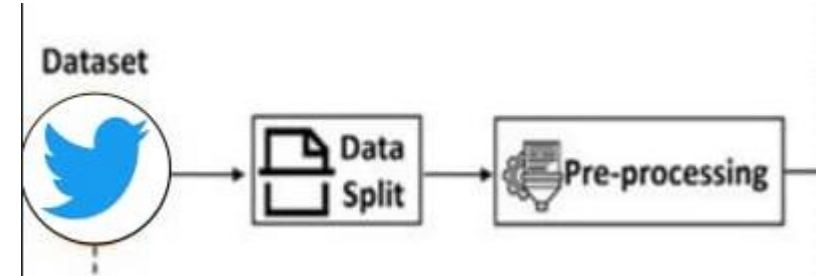


Methodology Used





1.Data Collection & Preprocessing



1. Dataset Sources: Twitter API, Facebook comments, YouTube reviews, Kaggle sentiment datasets.

2. Preprocessing Steps:

1.Text Cleaning: Remove special characters, URLs, and stopwords.

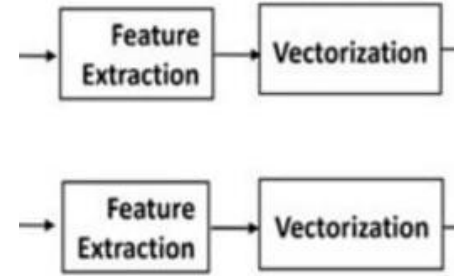
2.Lemmatization: Convert words to base form (e.g., "running" → "run").

3.Handling Emojis & Slang: Convert them into sentiment scores.

4.Dealing with Code-Mixed Language: Use transliteration techniques.



2.Feature Extraction Techniques :



- The raw text in X (Formerly Twitter) Tweets is converted into numerical representations for machine learning using two primary methods:

1. Syntactic (Non-Semantic) Features:

- **Bag of Words (BoW):** Counts word occurrences but ignores order.
- **TF-IDF:** Weighs the importance of words based on frequency in the dataset.

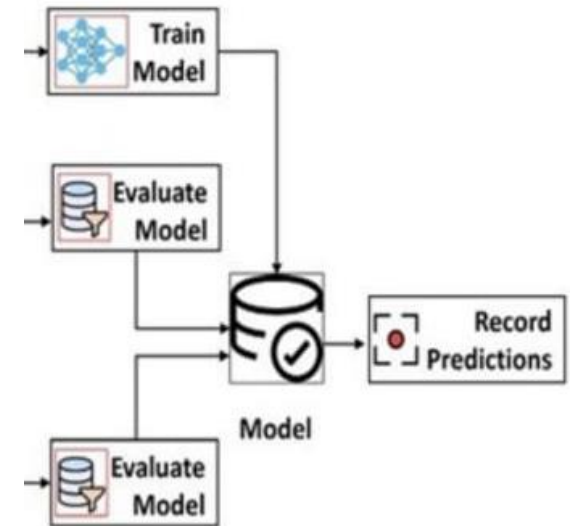
2. Semantic (Word Embeddings):

- **Word2Vec, GloVe, fastText:** Generate dense vector representations.
- **BERT, RoBERTa, GPT-based embeddings:** Contextual embeddings for improved semantic understanding.



3. Model Selection and Training

1. **Traditional ML Models:** SVM, Naïve Bayes, Random Forest.
2. **Deep Learning Models:** CNN, BiLSTM, Transformer models (BERT, RoBERTa).
3. **Hybrid Approaches:** CNN + LSTM combination for better context understanding.



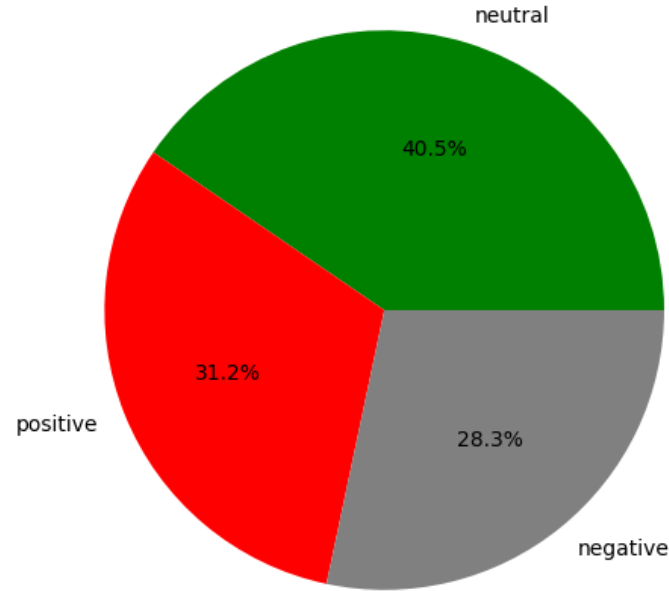
4. Performance Evaluation

1. **Classification Metrics:** Accuracy, Precision, Recall, F1-score.
2. **Robustness Testing:** Adversarial text evaluation (typos, sarcasm).
3. **Real-time Analysis:** Compare model predictions with trending topics.



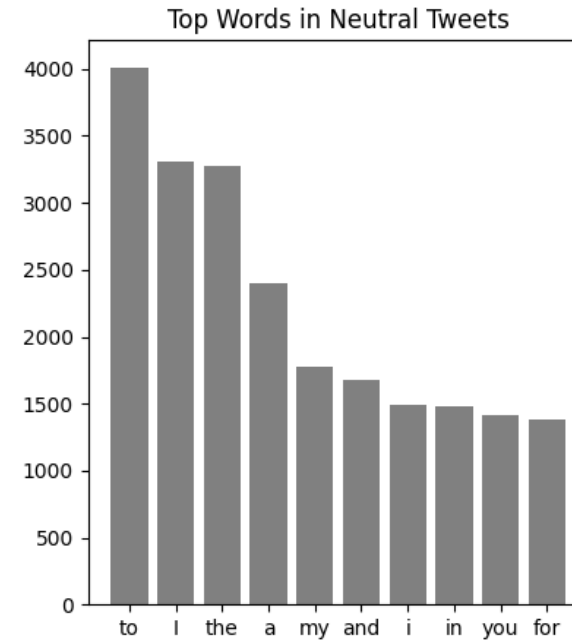
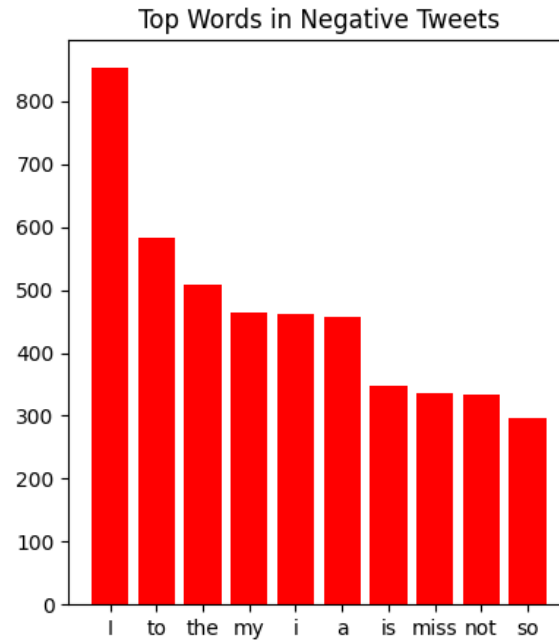
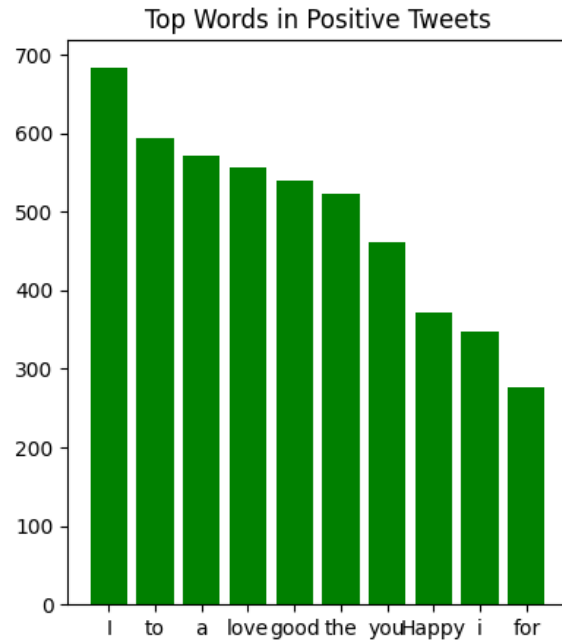
Key Findings While Preprocessing

Sentiment Distribution in Tweets Dataset



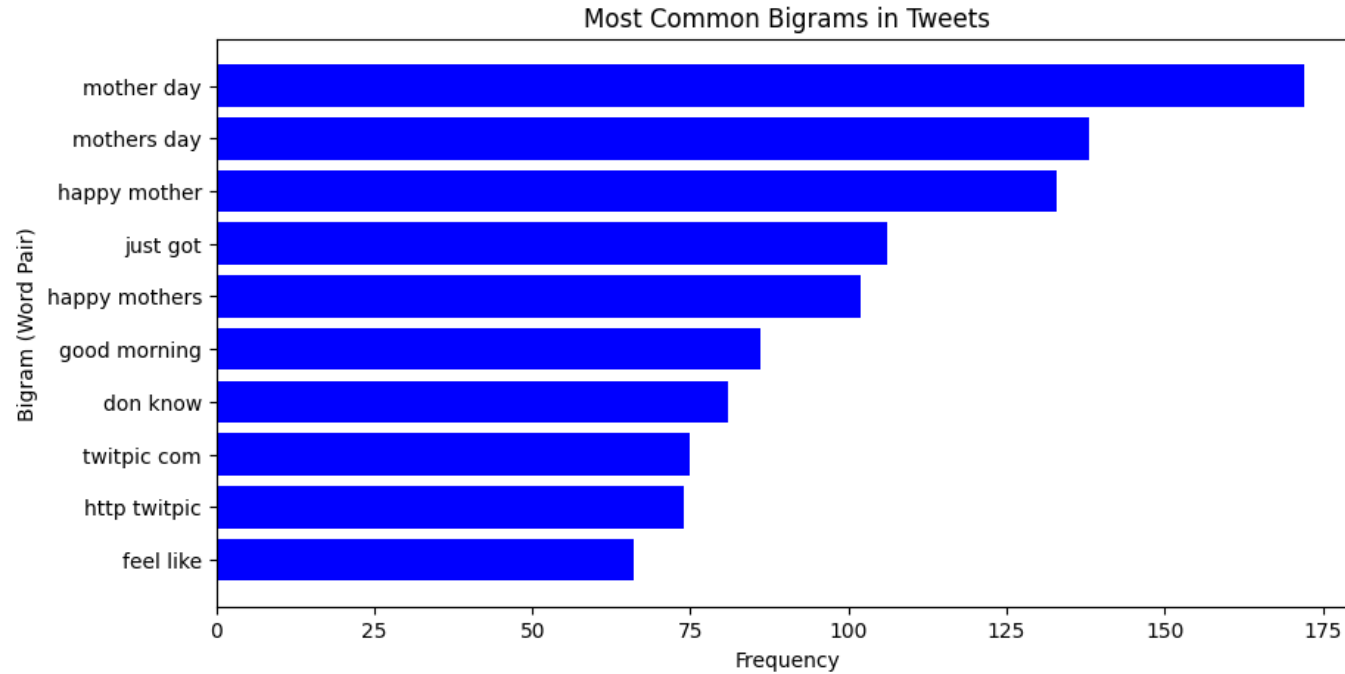
❖ Sentiment Distribution Pie Chart

- The pie chart displays the proportion of positive, negative, and neutral tweets in the dataset.
- Neutral tweets** make up the largest portion (40.5%), indicating that many tweets do not express strong emotions.
- Positive tweets** (31.2%) outnumber **negative tweets** (28.3%), suggesting a slight positivity bias in the dataset.
- This distribution is crucial for understanding the dataset's balance and ensuring the model does not become biased toward any single sentiment.



❖ Bar Charts for Top Words in Each Sentiment

- These charts show the most frequently used words in **positive, negative, and neutral** tweets.
- **Positive Words:** Words like "love," "good," "Happy," and "you" indicate emotions of joy and appreciation.
- **Negative Words:** Words such as "miss," "not," and "so" suggest expressions of sadness or dissatisfaction.
- **Neutral Words:** Common function words like "to," "the," and "my" appear frequently, indicating generic statements.



❖ Bigram Analysis (Most Common Word Pairs)

- The bigram bar chart highlights the most frequently occurring word pairs in tweets.
- Common phrases include "**mother day**," "**mothers day**," and "**happy mother**," indicating a significant presence of tweets related to Mother's Day.
- Other bigrams like "**good morning**" and "**feel like**" suggest daily conversations or personal expressions.



Explanation of Code

Importing Dependencies

```
import pandas as pd      # Data manipulation
import re                # Regular expressions for text cleaning
import nltk              # Natural Language Processing (NLP)
import numpy as np       # Numerical operations
import seaborn as sns    # Visualization (Confusion Matrix)
import matplotlib.pyplot as plt # Plotting graphs
from sklearn.feature_extraction.text import TfidfVectorizer # Feature extraction
from sklearn.model_selection import train_test_split # Data splitting
from sklearn.naive_bayes import MultinomialNB # Naïve Bayes Model
from sklearn.svm import SVC # Support Vector Machine
from sklearn.ensemble import RandomForestClassifier # Random Forest
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix # Model evaluation

nltk.download('stopwords')
nltk.download('wordnet')
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

Explanation:

- This block imports necessary libraries for data handling, text processing, ML models, and performance evaluation.
- NLTK downloads stopwords and WordNet lemmatizer, which help clean the text data.



Loading Dataset

```
import pandas as pd # Load and manipulate dataset
import numpy as np  # Handle numerical values if needed

# Step 1: Load the Dataset
data = pd.read_csv("/content/Tweets.csv")
```

Explanation:

- The dataset is loaded using `pandas.read_csv()`.
- This dataset contains **tweets with sentiment labels (positive, negative, neutral)**.



Data Preprocessing

```
import re # Regular expressions for text cleaning
import nltk # NLP processing
from nltk.corpus import stopwords # Stopword removal
from nltk.stem import WordNetLemmatizer # Lemmatization

# Step 2: preprocessing
def preprocess_text(text):
    if isinstance(text, str):
        lemmatizer = WordNetLemmatizer()
        stop_words = set(stopwords.words('english'))
        text = re.sub(r'^a-zA-Z\s', '', text) # Remove special characters
        words = text.lower().split()
        words = [lemmatizer.lemmatize(word) for word in words if word not in stop_words]
        return ' '.join(words)
    else:
        return ''

data['cleaned_text'] = data['text'].apply(preprocess_text)
```

Explanation:

- **Removes special characters** (`re.sub(r'^a-zA-Z\s', '', text)`).
- **Converts text to lowercase** (`text.lower()`).
- **Removes stopwords** like "the," "is," "and" (`stop_words = set(stopwords.words('english'))`).
- **Applies lemmatization** (`lemmatizer.lemmatize(word)`) to reduce words to their root forms.
- The cleaned text is stored in a new column '**cleaned_text**'.



Splitting Data

```
from sklearn.model_selection import train_test_split # Splitting dataset

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Explanation:

- Splits the dataset into **80% training data** and **20% testing data** using `train_test_split()`.
- **`random_state=42`** ensures reproducibility of results.



Feature Extraction

```
from sklearn.feature_extraction.text import TfidfVectorizer # TF-IDF transformation

# Step 4: Feature Extraction using TF-IDF
vectorizer = TfidfVectorizer(max_features=5000, stop_words='english', ngram_range=(1,2))
X = vectorizer.fit_transform(data['cleaned_text'])
y = data['sentiment'] # Sentiment labels (positive, negative, neutral)
```

Explanation:

- Converts text into numerical vectors using **TF-IDF (Term Frequency-Inverse Document Frequency)**.
- **Removes stop words** and extracts the top **5000 most important words**.
- Uses **n-grams (1,2)** to consider both individual words and word pairs.



Model Building and Training

```
from sklearn.naive_bayes import MultinomialNB # Naïve Bayes
from sklearn.svm import SVC # Support Vector Machine
from sklearn.ensemble import RandomForestClassifier # Random Forest

# Step 5: Model Selection & Training
models = {
    'Naïve Bayes': MultinomialNB(),
    'SVM': SVC(kernel='linear'),
    'Random Forest': RandomForestClassifier(n_estimators=100)
}

for model_name, model in models.items():
    model.fit(X_train, y_train)
```

Explanation:

- Three machine learning models are used:
 1. **Naïve Bayes** – Works well with text data.
 2. **Support Vector Machine (SVM)** – Performs well in high-dimensional space.
 3. **Random Forest** – An ensemble model for better generalization.
- **Trains each model** using `model.fit(X_train, y_train)`.



Performance Evaluation

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix # Evaluation metrics
import seaborn as sns # Confusion matrix visualization
import matplotlib.pyplot as plt # Graphs

# Step 6: Model Evaluation
for model_name, model in models.items():
    y_pred = model.predict(X_test)

    # Compute Accuracy
    accuracy = accuracy_score(y_test, y_pred) * 100

    # Compute Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)

    # Display Results
    print(f"Performance of {model_name}:")
    print(f"Accuracy: {accuracy:.2f}%")
    print("Confusion Matrix:")
    print(cm)
    print("Classification Report:")
    print(classification_report(y_test, y_pred))

    # Plot Confusion Matrix
    plt.figure(figsize=(6,5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=model.classes_, yticklabels=model.classes_)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(f"Confusion Matrix for {model_name}")
    plt.show()
```

Explanation:

- **Predicts sentiment for test data** using `model.predict(X_test)`.
- **Computes accuracy** using `accuracy_score()`.
- **Generates a confusion matrix** (`confusion_matrix(y_test, y_pred)`) to compare predicted vs. actual sentiment labels.
- **Generates a classification report** (`classification_report(y_test, y_pred)`) to display Precision, Recall, and F1-score.
- **Plots a heatmap of the confusion matrix** using Seaborn



Final Result

TWEETS DATASET	NAÏVE BAYES	RANDOM FOREST	SVM
1	63.98%	68.72%	69.78%
2	70.33%	87.58%	75.83%
3	82.08	81%	82.97%
4	72.96%	85.86%	84.39%

- These Results tells about the the performance (Accuracy) of the models on different datasets .
- Random Forest performs Very Good on most of the datasets .SVM and Naïve Bayes also comparatively good .



References

- [1] B. Liu, *Sentiment Analysis: Mining Opinions, Sentiments, and Emotions*. Cambridge, UK: Cambridge University Press, 2015.
- [2] B. Pang and L. Lee, “Opinion Mining and Sentiment Analysis,” *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008.
- [3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2021.
- [4] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [5] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [6] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [7] Y. Goldberg, *Neural Network Methods for Natural Language Processing*. Morgan & Claypool Publishers, 2017.